

Exercício em sala — Sistema de Pagamentos com S e O

Crie um pequeno sistema de pagamentos para uma loja.

O sistema deve permitir inicialmente:

- PIX
- Cartão
- Boleto

Cada forma de pagamento deve possuir uma maneira própria de processamento.

Parte 1 — Aplicando o S

O princípio **S — Single Responsibility Principle** diz que cada classe deve ter uma responsabilidade principal.

Por isso, não coloque tudo dentro da mesma classe.

Crie uma classe responsável pelos dados básicos do pagamento:

Python

```
class Pagamento:
    def __init__(self, valor):
        self.valor = valor
```

Depois, cada tipo de pagamento ficará responsável apenas por sua forma de processamento.

Exemplo:

Python

```
class PagamentoPix:
    def __init__(self, valor):
        self.valor = valor

    def processar(self):
        print(f"PIX de R$ {self.valor:.2f} processado.")
```

Além disso, crie classes separadas para outras responsabilidades.

ComprovanteService

Responsável somente por gerar o comprovante:

Python

```
class ComprovanteService:

    def gerar(self, pagamento):
        print(f"Comprovante gerado no valor de R$
        {pagamento.valor:.2f}")
```

PagamentoRepository

Responsável somente por salvar o pagamento:

Python

```
class PagamentoRepository:

    def salvar(self, pagamento):
        print(f"Pagamento de R$ {pagamento.valor:.2f} salvo.")
```

Assim temos:

None

Pagamento

↓

dados do pagamento

PagamentoPix / PagamentoCartao / PagamentoBoleto

↓

processamento

ComprovanteService

↓

comprovantes

PagamentoRepository

↓

persistência

Isso aplica o **S**.

Parte 2 — Aplicando o O

O princípio **O** — **Open/Closed Principle** diz:

O sistema deve estar aberto para extensão, mas fechado para modificação.

A ideia é que seja possível adicionar uma nova forma de pagamento sem precisar alterar as classes já existentes.

Por exemplo, depois que PIX, cartão e boleto estiverem prontos, a empresa pede suporte a PayPal.

Você deve conseguir apenas criar:

Python

```
class PagamentoPayPal:

    def __init__(self, valor):
        self.valor = valor

    def processar(self):
        print(f"Pagamento PayPal de R$ {self.valor:.2f}
        processado.")
```

Sem alterar:

Python

```
PagamentoPix
PagamentoCartao
PagamentoBoleto
```

```
ComprovanteService  
PagamentoRepository
```

Estrutura esperada

```
Python  
PagamentoPix  
PagamentoCartao  
PagamentoBoleto  
PagamentoPayPal  
  
ComprovanteService  
  
PagamentoRepository
```

Cada pagamento possui:

```
Python  
def processar(self):  
    ...
```

Mas cada um implementa seu próprio comportamento.

Enunciado

Sistema de Pagamentos

Uma loja deseja criar um pequeno sistema para processar diferentes formas de pagamento.

Crie as classes:

```
Python
PagamentoPix
PagamentoCartao
PagamentoBoleto
```

Cada classe deve possuir:

```
Python
valor
```

e o método:

```
Python
processar()
```

O método deve exibir uma mensagem diferente para cada forma de pagamento.

Exemplo:

```
None
PIX de R$ 100.00 processado.

Pagamento no cartão de R$ 250.00 processado.

Boleto de R$ 500.00 gerado.
```

Crie também uma classe:

```
Python
ComprovanteService
```

com o método:

```
Python
gerar(pagamento)
```

Ela deve ser responsável somente pela geração do comprovante.

Crie ainda:

```
Python
PagamentoRepository
```

com o método:

```
Python
salvar(pagamento)
```

Ela deve ser responsável somente por simular o salvamento do pagamento.

Segunda etapa

Depois que o sistema estiver funcionando, adicione uma nova forma de pagamento:

```
Python
PagamentoPayPal
```

Ela deve possuir seu próprio método:

```
Python
processar()
```

O aluno **não deve modificar as classes anteriores** para adicionar o PayPal.

Exemplo de uso esperado

Python

```
pix = PagamentoPix(100)
cartao = PagamentoCartao(250)
boleto = PagamentoBoleto(500)

pix.processar()
cartao.processar()
boleto.processar()

comprovante = ComprovanteService()
repository = PagamentoRepository()

comprovante.gerar(pix)
repository.salvar(pix)
```

Depois:

Python

```
paypal = PagamentoPayPal(300)

paypal.processar()

comprovante.gerar(paypal)
repository.salvar(paypal)
```

O que precisa identificar

Onde está o S?

Em:

None

Pagamento → processamento
ComprovanteService → comprovante

PagamentoRepository → salvamento

As responsabilidades estão separadas.

Onde está o O?

Quando adicionamos:

Python

```
class PagamentoPayPal:
```

sem modificar PIX, cartão ou boleto.